
StarSync Network Brief

Release 0.0.0

Author(s): DATRO Consortium

Apr 07, 2026

CONTENTS

1	Document Information	1
1.1	Release Notes	1
1.2	Executive Summary	1
1.3	Proposed Solution	1
1.4	Work Packages	3
1.5	Timeline	3
1.6	Deliverables	3
1.7	Risks and Mitigations	4
1.8	Approval	4
1.9	Document Author(s):	4

DOCUMENT INFORMATION

1.1 Release Notes

Version	Date	Description
0.0.0	Apr 07, 2026	Initial brief for StarSync Network project

1.1.1 0.0.0 (Apr 07, 2026)

Initial Release

Documents the proposed solution for the StarSync Network, including architecture, work packages, and implementation approach for automated repository synchronization across three machines with GitHub as the central hub.

1.2 Executive Summary

This document proposes the detailed approach for delivering the StarSync Network project. The project addresses the need for automated repository synchronization between multiple development endpoints (laptop, AWS-command, AWS-ai) using GitHub as the central hub in a star network topology.

The proposed solution implements a cron-based synchronization script that runs every 5 minutes on each endpoint. The script detects local changes, commits and pushes them to autosync branches on GitHub, creates pull requests for client review, monitors for remote changes, and pulls updates automatically. When failures occur, a Hermes agent is spawned in YOLO mode for autonomous recovery.

The architecture consists of three layers:

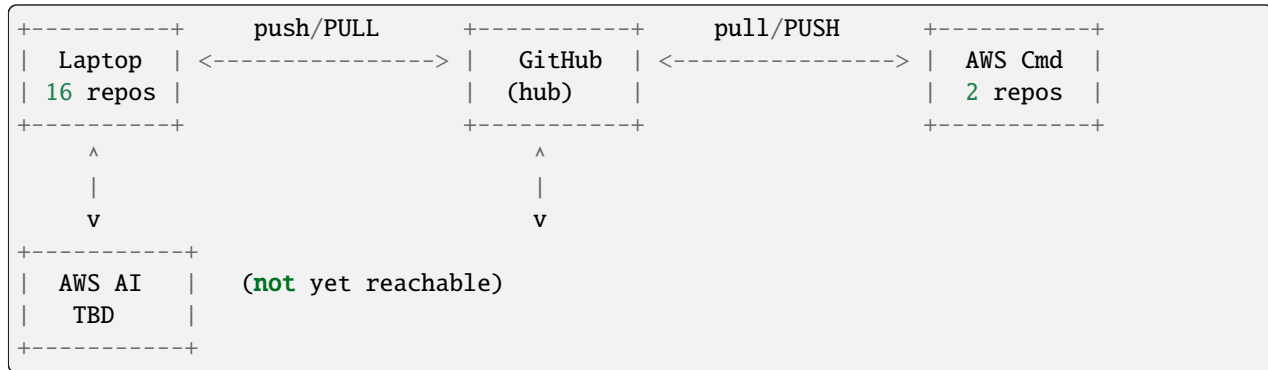
- **Layer 1: Endpoints** – Each machine runs the sync script via cron
- **Layer 2: GitHub** – Acts as the central hub, receiving pushes via autosync branches and hosting pull requests for review
- **Layer 3: Notifications** – Telegram delivers PR URLs to the client for review

Success will be measured by automatic detection and propagation of changes across all connected endpoints, with PRs created for review before integration.

1.3 Proposed Solution

1.3.1 Architecture

The synchronization system uses a star network topology with GitHub at the center.



1.3.2 Component Design

1. Sync Script (repo-sync.sh)

A bash script deployed on each endpoint that:

- Reads a list of tracked repositories from a config file
- For each repo: - Counts uncommitted changes (`git status --porcelain`) - Fetches from origin to check for remote updates - If behind origin: stashes local changes, pulls with rebase, restores stash - If changes exist and count ≤ 500 : commits, pushes to an autosync branch - Creates a PR via `gh pr create` - Sends a Telegram notification with the PR URL
- Returns exit code 0 (ok), 1 (failures), or 2 (nothing to do)

2. Telegram Notification

Upon PR creation, the script uses the Telegram Bot API to send an HTML-formatted message containing:

- The origin hostname
- Repository name
- Number of files changed
- A clickable link to the PR on GitHub

3. Agent Fallback (repo-sync-agent.sh)

When the cron detects a non-zero exit code from the sync script, it invokes a Hermes agent using the `reposync` profile in YOLO mode with the instruction to diagnose and resolve the failure. The agent:

- Reads the latest sync log from `/tmp/repo-sync/`
- Attempts automated resolution (abort, reset, conflict handling)
- Reports results via Telegram
- This enables self-healing sync operations without human intervention

4. Cron Orchestration

Each endpoint has a cron entry running every 5 minutes:

```
*/5 * * * * /path/to/repo-sync-cron.sh >> /tmp/repo-sync/cron.log 2>&1 || /path/to/repo-sync-agent.sh >> /tmp/repo-sync/cron.log 2>&1
```

The `||` operator ensures the agent fallback runs only when the sync script exits with a non-zero code (failures).

5. Repository Tracking

A simple text file (`reposync_repos.txt`) lists full paths to all tracked repositories on each endpoint. Lines starting with `#` are comments, empty lines are ignored. This file is machine-specific.

1.3.3 Data Flow

1. User makes changes in a repo on **laptop**
2. Cron runs (within 5 min): detects changes, commits, pushes
3. Push creates `autosync/laptop/20260407-XXXXXX` branch on GitHub
4. PR is created targeting `gh-pages`
5. Telegram message sent to client with PR URL
6. Client reviews at the GitHub PR page and merges
7. Next cron cycle on **aws-command** pulls the merged changes
8. All endpoints are now in sync

1.4 Work Packages

The project is decomposed into six discrete work packages:

WPI is the primary work package, encompassing script development, cron configuration, repository list creation, and environment setup on the laptop.

1.5 Timeline

Phase 1: Infrastructure (Minutes 1-5) - Write and deploy sync script on laptop - Create repository list - Install cron job

Phase 2: Remote Deployment (Minutes 5-10) - SSH to aws-command, deploy sync infrastructure - Install cron job - Test connectivity

Phase 3: static/ui Push (Minutes 10-15) - Connect to aws-command - Commit static/ui folder - Push to GitHub, create PR - Send Telegram notification

Phase 4: Documentation (Minutes 15-30) - Write Mandate, Brief, Plan, Highlight Report - Build HTML and PDF - Create PR with preview link

Phase 5: Verification (Minutes 30-45) - Verify cron is active - Verify Telegram notifications deliver - Test agent fallback trigger

1.6 Deliverables

1. **repo-sync.sh** – Sync script deployed on laptop and aws-command
2. **repo-sync-agent.sh** – Agent fallback wrapper script on laptop
3. **reposync_repos.txt** – Tracked repository list on both endpoints
4. **Cron Jobs** – Active on both endpoints (every 5 minutes)
5. **Telegram Notifications** – Tested and working, delivering PR URLs
6. **PR #264** – static/ui folder pushed from aws-command to GitHub

7. **Documentation** – Mandate, Brief, Plan, and Highlight Report compiled to HTML and PDF in the Datro Consortium Document Library

1.7 Risks and Mitigations

1.8 Approval

Role	Name
Project Sponsor Senior User Senior Supplier	Client (unclehowell) Client (unclehowell) DATRO Consortium - AI

1.8.1 Sign-off

Role	Date	Signature
Project Sponsor Senior User Senior Supplier		

1.9 Document Author(s):

DATRO Consortium - Second Brain (AI)